EXAM-DAY CHEAT SHEET - Python (print this, 3-4 pages)

Everything in the code blocks is one continuous, tested script (pandas -> charts -> SQLite -> CSV/numpy -> PyTorch). Change the names, keep the shape. Detail and explanations: Paper_Solutions/P1 and Topics/T1-T8.

Before you type: exact names? (file `your_name_python_task.py`, DB, table, columns) - `head()`, `isna().sum()`, `duplicated().sum()`, `dtypes` - run after every sub-task - print every answer.

A. Imports and loading

```
import sqlite3, os, csv
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim

df = pd.read_csv("sales_data.csv")                  # Task: load
print(df.shape, df.dtypes, df.isna().sum(), df.duplicated().sum(), sep="\n")
```

B. Cleaning (assign back every time)

```
df["Product"] = df["Product"].fillna("Unknown")                  # text blank
df["Price"] = df["Price"].fillna(df["Total"] / df["Quantity"])   # recover
df["Price"] = df["Price"].fillna(0)                              # fallback
df["Total"] = df["Total"].fillna(df["Quantity"] * df["Price"])
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)   # dd/mm/yyyy
df["Total"] = df["Quantity"] * df["Price"]               # fix discrepancies
df = df.drop_duplicates()                                # remove copies
# other fills: df["x"].fillna(df["x"].mean()) / .median() / .mode()[0]
# other date format: pd.to_datetime(df["Date"], format="%m/%d/%Y")
# drop rows instead: df = df.dropna(subset=["Price"])
```

C. Group, filter, charts

```
d23 = df[df["Date"].dt.year == 2023]                      # filter year
qty = df.groupby("Product")["Quantity"].sum()             # pile + add
month = d23.groupby(d23["Date"].dt.month)["Total"].sum()  # 2023 by month

plt.figure(figsize=(8, 5))                 # NEW paper for EVERY chart
plt.bar(qty.index, qty.values)             # bar
plt.title("Title"); plt.xlabel("X"); plt.ylabel("Y")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()

plt.figure(); plt.plot(month.index, month.values, marker="o")   # line
plt.title("Sales 2023"); plt.xlabel("Month"); plt.ylabel("Total"); plt.show()
# histogram: plt.hist(values, bins=5)   pie: plt.pie(v, labels=l, autopct="%1.1f%%")
# two plots: fig, (a, b) = plt.subplots(1, 2); a.bar(...); b.hist(...)
# save instead of show: plt.savefig("chart.png")
```

D. SQLite

```
conn = sqlite3.connect("SalesDB.db"); cur = conn.cursor()
cur.execute("""CREATE TABLE IF NOT EXISTS Sales (
                   Date TEXT, Product TEXT, Quantity INTEGER,
                   Price REAL, Total REAL,
                   UNIQUE (Date, Product, Quantity, Price, Total))""")
out = df.copy(); out["Date"] = out["Date"].dt.strftime("%Y-%m-%d")
for _, r in out.iterrows():
    cur.execute("INSERT OR IGNORE INTO Sales VALUES (?, ?, ?, ?, ?)",
```

```
                (r["Date"], r["Product"], int(r["Quantity"]),
                 float(r["Price"]), float(r["Total"])))
conn.commit()

cur.execute("SELECT SUM(Total) FROM Sales WHERE strftime('%Y', Date) = '2023'")
print("Total 2023:", cur.fetchone()[0])
cur.execute("""SELECT Product, SUM(Quantity) AS q FROM Sales
               WHERE strftime('%Y', Date) = '2023'
               GROUP BY Product ORDER BY q DESC""")
for product, q in cur.fetchall():
    print(product, q)
# one row:  fetchone()   all rows: fetchall()   count: SELECT COUNT(*)
# DELETE FROM Sales WHERE ...;  DELETE FROM Sales;   UPDATE Sales SET Price=? WHERE ...
# dup check first: SELECT 1 FROM T WHERE k = ?  -> fetchone() is not None
# to DataFrame: pd.read_sql_query("SELECT ...", conn)
conn.close()
```

E. CSV file: create-or-append, input loop, numpy summary

```
FILE = "heights.csv"
if not os.path.exists(FILE):
    with open(FILE, "w", newline="") as f:
        csv.writer(f).writerow(["Name", "Height"])
    print("heights.csv created")
else:
    print("heights.csv exists - appending")

for name, h in [("Amy", "165"), ("Ben", "172"), ("q", "")]:   # replace with input()
    if name.lower() == "q":                    # name = input("Name (q to quit): ")
        break
    try:
        height = int(h)                        # h = input("Height (cm): ")
    except ValueError:
        print("Height must be a whole number"); continue
    with open(FILE, "a", newline="") as f:     # a = append
        csv.writer(f).writerow([name, height])

h = pd.read_csv(FILE)["Height"].to_numpy()
text = (f"Records: {len(h)}\nAverage: {np.mean(h):.1f}\n"
        f"Min: {np.min(h)}\nMax: {np.max(h)}\n")
print(text)
with open("height_summary.txt", "w") as f:
    f.write(text)
plt.figure(); plt.hist(h, bins=5)
plt.title("Height Distribution"); plt.xlabel("Height (cm)"); plt.ylabel("Frequency")
plt.show()
```

F. PyTorch (spec: 10 -> 5 ReLU -> 3 ReLU -> 1 Sigmoid, MSE, SGD 0.01, 20 epochs)

```
class SimpleNN(nn.Module):
    def __init__(self):
        super().__init__()                     # FIRST line, always
        self.fc1 = nn.Linear(10, 5)            # out of one = in of next
        self.fc2 = nn.Linear(5, 3)
        self.fc3 = nn.Linear(3, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return torch.sigmoid(self.fc3(x))

model = SimpleNN()
criterion = nn.MSELoss()                       # classification: nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)   # or optim.Adam
data = torch.randn(100, 10); target = torch.randn(100, 1)   # (100,1) not (100,)
losses = []
for epoch in range(20):
```

```
    optimizer.zero_grad()                       # 1 wipe
    loss = criterion(model(data), target)       # 2+3 forward, loss
    loss.backward()                             # 4 blame
    optimizer.step()                            # 5 adjust
    losses.append(loss.item())                  # number, not tensor
plt.figure(); plt.plot(range(1, 21), losses)
plt.title("Training loss"); plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.show()
# accuracy (binary): ((model(data) > 0.5).float() == target).float().mean().item()
```

G. Python reminders

```
f"{x:.2f}" 2 decimals    {x:>8} right-align   {name:<10} left-align
int(input("n: "))        try: ... except ValueError: ...      while True: ... break
for i, v in enumerate(lst):      dict.get(k, 0)      sorted(d.items(), key=lambda kv: kv[1], reverse=True)
class A: def __init__(self, x): self.x = x      def __str__(self): return f"A({self.x})"
and / or / not      == (compare) vs = (assign)       lst[1:4]  s[::-1]   len(x)
```

H. Error -> cause -> fix

You see	Cause	Fix
`ValueError: time data ... doesn't match`	day/month order	`dayfirst=True` or `format=`
`AttributeError: Can only use .dt accessor`	Date still text	convert with `to_datetime` first
`no such table`	queried before CREATE / wrong folder	create table first; check cwd
`UNIQUE constraint failed`	inserting duplicate with plain INSERT	`INSERT OR IGNORE`
`Error binding parameter` / BLOB junk	numpy/Timestamp into SQLite	`int()`, `float()`, `strftime`
`database is locked`	an old connection still open	`conn.close()`, restart kernel
`mat1 and mat2 shapes cannot be multiplied`	layer sizes do not chain	out of one = in of next
`AttributeError: cannot assign module before Module.__init__()`	missing `super().__init__()`	add it first
`KeyError: 'Total'`	wrong column name / spaces	`print(df.columns)`
`SettingWithCopyWarning`	editing a filtered slice	`.copy()` the slice first
`FileNotFoundError`	CSV not beside the script	check `os.getcwd()`

I. Last 10 minutes

- [ ] delete `.db`, run the **whole file** from a clean start - no errors
- [ ] every chart: title + both axis labels - every answer printed with a label
- [ ] comments quote the question; names exact; file named `your_name_python_task.py`
- [ ] saved, uploaded, checked the upload shows the right file